# Mini-Project 2: Process Layout and Creation Report

## 1. Introduction

The objective of this project is to explore the abstraction layer provided by the Operating System through the concept of a "Process." We investigated how the OS manages memory segments and the system calls used to create and transform processes.

---

## 2. Part I: The Process Memory Layout

Every process has a specific memory layout consisting of different segments. We verified this by examining the `/proc/[PID]/maps` file while running a custom C program.

### Key Memory Segments Observed:

- **Code (Text) Segment:** Contains the executable instructions. We located this by printing the address of a code label (`&&foo`).
- **Data Segment:** Stores global variables.
- **Heap:** Used for dynamic memory allocation.
- **Stack:** Stores local variables and function call information. It grows downwards in memory.

**Observation:** Using the `maps` file, we confirmed that the addresses generated by our program fall within the ranges allocated by the Linux kernel.

---

## 3. Part II: Process Creation (fork)

We used the `fork()` system call to create a new process. `fork()` creates an exact duplicate of the parent process.

### Experimental Results:

- **Parent Process:** Received the PID of the child.

- **Child Process:** Received a return value of `0`.
- **Outcome:** We successfully observed both processes executing simultaneously, printing their unique Identifiers (PIDs).

---

# 4. Part III: The Zombie State

A "Zombie" process is a process that has completed execution but still has an entry in the process table because its parent has not yet read its exit status.

### Experimental Steps:

1. Created a child process that exits immediately.
2. Made the parent process `sleep()` for 60 seconds without calling `wait()`.
3. **Result:** Running the `ps -l` command showed the child process in a **'Z' (Zombie)** state, labeled as `<defunct>`.

---

# 5. Part IV: Process Transformation (exec)

We explored the `exec()` system call, which allows a process to overwrite its own image with a new program.

### Experimental Result:

We used `execlp()` to transform our running program into the `ls` command. The PID remained the same, but the entire memory space (Code, Data, Stack) was replaced by the `ls` program code.

---

# 6. Conclusion

Through these hands-on experiments in the Linux terminal, we successfully mapped the theoretical concepts of Operating Systems to practical observations. We confirmed the process memory structure, the cloning mechanism of `fork()`, the lifecycle of a process, and the power of process transformation.

**Golden Rule of Engineering:** *"If In Doubt, Try It Out!"*